Team Delta API Contract

Team Delta owns the **School Administration backend** for Hatchloom -- the services that power school admin screens for managing experiences, cohorts, student enrolments, and the admin dashboard.

This document describes every HTTP endpoint exposed by Team Delta's three microservices. It is intended for other Hatchloom teams (especially **Team Romeo**, who builds student-facing dashboards) to understand how to consume our APIs.

Current version: D2 (March 2026) -- most D1 limitations resolved; remaining items depend on cross-team integration.

Authentication

All protected endpoints require a `Authorization: Bearer {token}` header.

For D1, authentication is mocked. Three hardcoded tokens are recognized:

Token	User ID	Role	School ID
`test-admin-token`	1	`school_admin`	1
`test-teacher-token`	2	`school_teacher`	1
`test-student-token`	4	`student`	1

Behavior:

- Missing or empty `Authorization` header returns `401 Unauthenticated`.
- Unrecognized token returns `401 Unauthenticated`.
- Valid token mapping to a user whose role is not `school_admin` or `school_teacher` returns `403 Forbidden`.

Integration dependency: This is mock auth only. Real JWT/token-based authentication will come from Team Quebec's Auth service. See the "Integration Requests" section below for details.

Services Overview

Service	Port	Owns (Tables)	Description
Dashboard Service	8001	None (aggregation only)	Screen 300 -- admin dashboard, widgets, reporting
Experience Service	8002	`experiences`, `experience_courses`	Screens 301/302 -- experience CRUD and detail views
Enrolment Service	8003	`cohorts`, `cohort_enrolments`	Screen 303 -- cohort management and student enrolment

All three services share the same PostgreSQL database and the same `schools`/`users` reference tables (seeded as mock data).

Enrolment Service (port 8003)

Base URL: `http://localhost:8003/api/school`

Health Check

```
GET /api/school/enrolments/health
```

No authentication required.

Response (200):

```
{
  "status": "ok",
  "service": "enrolment",
  "timestamp": "2026-03-12T14:30:00+00:00"
}
```

List Cohorts

```
GET /api/school/cohorts
```

Headers:

- `Authorization: Bearer {token}` (required)

Query Parameters:

Parameter	Type	Required	Description
`experience_id`	integer	No	Filter cohorts by experience
`status`	string	No	Filter by status: `not_started`, `active`, `completed`
`search`	string	No	Case-insensitive partial match on cohort name

Response (200):

```
{
  "data": [
    {
      "id": 1,
      "name": "Spring 2026 Cohort A",
      "experience_id": 1,
      "status": "active",
      "teacher_name": "Ms. Johnson",
      "student_count": 12,
      "removed_count": 2,
      "capacity": 30,
      "start_date": "2026-03-01",
      "end_date": "2026-06-15"
    },
    {
      "id": 2,
      "name": "Fall 2026 Cohort B",
```

```
      "experience_id": 1,
      "status": "not_started",
      "teacher_name": null,
      "student_count": 0,
      "removed_count": 0,
      "capacity": 25,
      "start_date": "2026-09-01",
      "end_date": "2026-12-15"
    }
  ]
}
```

Notes:

- `student_count` reflects only actively enrolled students (not removed ones).
- `removed_count` reflects the number of students who were removed from the cohort.
- Results are automatically scoped to the authenticated user's school.

Create Cohort

```
POST /api/school/cohorts
```

Headers:

- `Authorization: Bearer {token}` (required)
- `Content-Type: application/json`

Request Body:

Field	Type	Required	Description
`experience_id`	integer	Yes	Must reference an existing experience
`name`	string	Yes	Max 255 characters
`start_date`	date (YYYY-MM-DD)	Yes	Must be today or later
`end_date`	date (YYYY-MM-DD)	Yes	Must be after start_date
`capacity`	integer	No	Minimum 1 if provided
`teacher_id`	integer	No	Must reference an existing user

Response (201):

```
{
  "id": 3,
  "name": "Winter 2026 Cohort C",
  "experience_id": 1,
  "status": "not_started",
  "capacity": 20,
  "start_date": "2026-01-10",
  "end_date": "2026-04-20",
  "created_at": "2026-03-12T14:30:00+00:00"
}
```

Error (422) -- Validation failure:

```
{
  "message": "The experience id field is required.",
  "errors": {
```

```
    "experience_id": ["The experience id field is required."]
  }
}
```

Note: New cohorts are always created with status `not_started`.

Show Cohort

```
GET /api/school/cohorts/{id}
```

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "id": 1,
  "name": "Spring 2026 Cohort A",
  "experience_id": 1,
  "status": "active",
  "teacher_name": "Ms. Johnson",
  "student_count": 12,
  "removed_count": 2,
  "capacity": 30,
  "start_date": "2026-03-01",
  "end_date": "2026-06-15"
}
```

Error (404):

```
{
  "error": true,
  "message": "Cohort not found",
  "code": "NOT_FOUND"
}
```

Update Cohort

```
PUT /api/school/cohorts/{id}
```

Headers:

- `Authorization: Bearer {token}` (required)
- `Content-Type: application/json`

Request Body (all fields optional):

Field	Type	Description
`name`	string	Max 255 characters
`start_date`	date (YYYY-MM-DD)	
`end_date`	date (YYYY-MM-DD)	Must be after start_date
`capacity`	integer	Minimum 1
`teacher_id`	integer	Must reference an existing user

Response (200):

```
{
  "id": 1,
  "name": "Spring 2026 Cohort A (Updated)",
```

```
  "experience_id": 1,
  "status": "active",
  "capacity": 35,
  "start_date": "2026-03-01",
  "end_date": "2026-06-30"
}
```

Activate Cohort

```
PATCH /api/school/cohorts/{id}/activate
```

Transitions a cohort from `not_started` to `active`. Only valid when current status is `not_started`.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "id": 1,
  "name": "Spring 2026 Cohort A",
  "status": "active"
}
```

Error (409) -- Invalid state transition:

```
{
  "error": true,
  "message": "Cohort is already active or completed",
  "code": "INVALID_STATE_TRANSITION"
}
```

Complete Cohort

```
PATCH /api/school/cohorts/{id}/complete
```

Transitions a cohort from `active` to `completed` (terminal state). Only valid when current status is `active`.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "id": 1,
  "name": "Spring 2026 Cohort A",
  "status": "completed"
}
```

Error (409) -- Invalid state transition:

```
{
  "error": true,
  "message": "Cohort must be active to complete",
  "code": "INVALID_STATE_TRANSITION"
}
```

List Enrolments (Student Overview)

```
GET /api/school/enrolments
```

Returns a paginated list of students with their cohort assignments and computed assignment status.

Headers:

- `Authorization: Bearer {token}` (required)

Query Parameters:

Parameter	Type	Required	Description
`search`	string	No	Case-insensitive partial match on student name
`per_page`	integer	No	Items per page (default: 15)
`grade`	string	No	Accepted but not applied (no grade column in users table yet)
`experience_id`	integer	No	Filter to students enrolled in cohorts of this experience
`cohort_id`	integer	No	Filter to students enrolled in this specific cohort
`student_id`	integer	No	Filter to a single student by ID

Response (200):

```
{
  "data": [
    {
      "student_id": 3,
      "name": "Alice Student",
      "email": "alice@school.test",
      "cohort_assignments": [
        {
          "cohort_id": 1,
          "cohort_name": "Spring 2026 Cohort A",
          "experience_name": "Entrepreneurship Basics",
          "status": "enrolled",
          "enrolled_at": "2026-03-05T10:00:00+00:00"
        }
      ],
      "assignment_status": "assigned"
    },
    {
      "student_id": 4,
      "name": "Bob Student",
      "email": "bob@school.test",
      "cohort_assignments": [],
      "assignment_status": "not_assigned"
    }
  ],
  "meta": {
    "current_page": 1,
    "last_page": 1,
    "per_page": 15,
    "total": 2
  }
}
```

Assignment status logic:

- `assigned` -- student has at least one enrolment with status `enrolled` in an active cohort
- `removed` -- student has enrolments but ALL of them have status `removed`
- `not_assigned` -- student has no enrolments or none that qualify

Note: The `grade` filter is accepted but has no effect because the `users` table does not yet have a grade column. The `student_id` filter is used internally by the Experience Service to look up individual students within an experience context.

Enrol Student

```
POST /api/school/cohorts/{cohortId}/enrolments
```

Headers:

- `Authorization: Bearer {token}` (required)
- `Content-Type: application/json`

Request Body:

Field	Type	Required	Description
`student_id`	integer	Yes	Must be a student in the same school

Validation chain (in order):

1. Cohort must exist (404 if not)

2. Student must exist, belong to the same school, and have role `student` (422 if not)

3. Cohort must be in `active` status (422 if not)

4. Cohort must not be at full capacity (422 if at capacity)

5. Student must not already be enrolled in this cohort, including removed enrolments -- re-enrolment is not allowed (422 with code `DUPLICATE_ENROLMENT`)

Response (201):

```
{
  "id": 5,
  "cohort_id": 1,
  "student_id": 3,
  "status": "enrolled",
  "enrolled_at": "2026-03-12T14:30:00+00:00"
}
```

Error (422) -- Cohort not active:

```
{
  "error": true,
  "message": "Cohort is not active",
  "code": "VALIDATION_ERROR"
}
```

Error (422) -- Duplicate:

```
{
  "error": true,
  "message": "Student is already enrolled in this cohort",
  "code": "DUPLICATE_ENROLMENT"
```

```
}
```

Error (422) -- At capacity:

```
{
  "error": true,
  "message": "Cohort is at full capacity",
  "code": "VALIDATION_ERROR"
}
```

Remove Student from Cohort

`DELETE /api/school/cohorts/{cohortId}/enrolments/{studentId}`

Soft-removes a student from a cohort (sets status to `removed` and records `removed_at` timestamp). Only removes enrolments with status `enrolled`.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "message": "Student removed from cohort"
}
```

Error (404) -- Enrolment not found:

```
{
  "error": true,
  "message": "Enrolment not found",
  "code": "NOT_FOUND"
}
```

Enrolment Statistics

`GET /api/school/enrolments/statistics`

Returns aggregate enrolment statistics for the school, including warnings.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "total_students": 25,
  "enrolled": 18,
  "assigned": 15,
  "not_assigned": 10,
  "removed": 3,
  "warnings": [
    {
      "type": "unassigned_students",
      "message": "10 students are not assigned to any active cohort",
      "severity": "warning"
    },
    {
      "type": "capacity_warning",
      "message": "Spring 2026 Cohort A is at 93% capacity (28/30)",
      "severity": "info"
    }
  ]
}
```

Field definitions:

- `total_students` -- all students in the school
- `enrolled` -- unique students with at least one `enrolled` enrolment (any cohort status)
- `assigned` -- unique students with an `enrolled` enrolment in an `active` cohort
- `not_assigned` -- `total_students - assigned`
- `removed` -- total number of enrolment records with status `removed`

Warning types:

- `unassigned_students` (severity: `warning`) -- triggered when any students lack an active cohort enrolment
- `capacity_warning` (severity: `info`) -- triggered when an active cohort is at 90%+ of its capacity

Student Detail

```
GET /api/school/enrolments/students/{studentId}
```

Returns detailed enrolment information for a single student, including all cohort assignments and credential summary.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "student": {
    "id": 3,
    "name": "Alice Student",
    "email": "alice@school.test",
    "grade": null
  },
  "enrolments": [
    {
      "cohort_id": 1,
      "cohort_name": "Spring 2026 Cohort A",
      "experience_name": "Entrepreneurship Basics",
      "status": "enrolled",
      "enrolled_at": "2026-03-05T10:00:00+00:00"
    }
  ],
  "credentials": {
    "total_earned": 0,
    "in_progress": 0,
    "details": []
  }
}
```

Error (404):

```
{
  "error": true,
  "message": "Student not found",
  "code": "NOT_FOUND"
}
```

Integration dependency: The `credentials` field is populated by a mock credential data provider (`MockCredentialDataProvider` implementing `CredentialDataProviderInterface`). It returns empty/zero data. Real credential data will come from Karl's credential engine. See the "Integration Requests" section below. The `grade` field is always `null` until the `users` table is extended with a grade column.

Export Enrolments (CSV)

```
GET /api/school/enrolments/export
```

Downloads a CSV file of all enrolment records for the school (including removed enrolments).

Headers:

- `Authorization: Bearer {token}` (required)

Response: Streamed CSV download (`Content-Type: text/csv`, filename: `enrolments.csv`)

CSV Columns:

Column	Description
`student_name`	Student's full name
`student_email`	Student's email address
`cohort_name`	Name of the cohort
`experience_name`	Name of the experience the cohort belongs to
`status`	`enrolled` or `removed`
`enrolled_at`	ISO 8601 timestamp
`removed_at`	ISO 8601 timestamp (empty if still enrolled)

Experience Service (port 8002)

Base URL: `http://localhost:8002/api/school`

Health Check

```
GET /api/school/experiences/health
```

No authentication required.

Response (200):

```
{
  "status": "ok",
  "service": "experience",
  "timestamp": "2026-03-12T14:30:00+00:00"
}
```

List Experiences

```
GET /api/school/experiences
```

Headers:

- `Authorization: Bearer {token}` (required)

Query Parameters:

Parameter	Type	Required	Description
`per_page`	integer	No	Items per page (default: 15)
`search`	string	No	Case-insensitive partial

			match on experience name

Response (200):

```
{
  "data": [
    {
      "id": 1,
      "name": "Entrepreneurship Basics",
      "description": "An introductory experience covering business fundamentals.",
      "status": "active",
      "course_count": 3,
      "cohort_count": 2,
      "created_by": "Ms. Johnson",
      "created_at": "2026-03-01T09:00:00+00:00"
    }
  ],
  "meta": {
    "current_page": 1,
    "last_page": 1,
    "per_page": 15,
    "total": 1
  }
}
```

Note: `cohort_count` is fetched from the Enrolment Service via HTTP. If the Enrolment Service is unavailable, `cohort_count` falls back to `0` (graceful degradation).

Create Experience

```
POST /api/school/experiences
```

Headers:

- `Authorization: Bearer {token}` (required)
- `Content-Type: application/json`

Request Body:

Field	Type	Required	Description
`name`	string	Yes	Max 255 characters
`description`	string	Yes	
`course_ids`	array of integers	Yes	At least one. Must be valid course IDs from the course catalogue

Response (201):

```
{
  "id": 2,
  "name": "Digital Marketing Fundamentals",
  "description": "Learn the basics of digital marketing and social media strategy.",
  "status": "active",
  "courses": [
    {
      "id": 101,
```

```
      "name": "Marketing 101",
      "sequence": 1
    },
    {
      "id": 102,
      "name": "Social Media Strategy",
      "sequence": 2
    }
  ],
  "created_at": "2026-03-12T14:30:00+00:00"
}
```

Error (422) -- Invalid course IDs:

```
{
  "error": true,
  "message": "One or more course IDs are invalid",
  "code": "VALIDATION_ERROR"
}
```

Integration dependency: Course IDs are validated against a `MockCourseDataProvider` (implementing `CourseDataProviderInterface`). The mock provider has a fixed set of valid course IDs. Real validation will use Team Papa's Course Service. See the "Integration Requests" section below.

Note: New experiences are always created with status `active`. The `courses` array in the response includes course names resolved from the (mock) course catalogue. Sequence is 1-based, derived from array position in `course_ids`.

Show Experience

```
GET /api/school/experiences/{id}
```

Returns a single experience with its courses and associated cohorts.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "id": 1,
  "name": "Entrepreneurship Basics",
  "description": "An introductory experience covering business fundamentals.",
  "status": "active",
  "courses": [
    {
      "id": 101,
      "name": "Business Planning",
      "sequence": 1
    },
    {
      "id": 102,
      "name": "Financial Literacy",
      "sequence": 2
    }
  ],
  "cohorts": [
    {
      "id": 1,
      "name": "Spring 2026 Cohort A",
      "status": "active",
      "student_count": 12
    }
  ],
```

```
  "created_by": "Ms. Johnson",
  "created_at": "2026-03-01T09:00:00+00:00"
}
```

Note: The `cohorts` array is fetched from the Enrolment Service over HTTP. If the Enrolment Service is unavailable, `cohorts` will be an empty array (graceful degradation).

Update Experience

```
PUT /api/school/experiences/{id}
```

Headers:

- `Authorization: Bearer {token}` (required)
- `Content-Type: application/json`

Request Body (all fields optional):

Field	Type	Description
`name`	string	Max 255 characters
`description`	string	
`course_ids`	array of integers	If provided, replaces all existing course associations (full replacement, not merge)

Response (200):

```
{
  "id": 1,
  "name": "Entrepreneurship Basics (Updated)",
  "description": "Updated description with new content.",
  "status": "active",
  "created_at": "2026-03-01T09:00:00+00:00"
}
```

Delete (Archive) Experience

```
DELETE /api/school/experiences/{id}
```

Soft-deletes the experience by setting its status to `archived` and then applying a soft delete.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "message": "Experience archived"
}
```

Experience Students (Screen 302)

```
GET /api/school/experiences/{id}/students
```

Returns individual student records enrolled in this experience, fetched from the Enrolment Service.

Headers:

- `Authorization: Bearer {token}` (required)

Query Parameters:

Parameter	Type	Required	Description
`search`	string	No	Case-insensitive partial match on student name or email
`per_page`	integer	No	Items per page (default: 15)

Response (200):

```
{
  "data": [
    {
      "student_id": 3,
      "student_name": "Alice Student",
      "student_email": "alice@school.test",
      "cohort_id": 1,
      "cohort_name": "Spring 2026 Cohort A",
      "status": "enrolled",
      "enrolled_at": "2026-03-05T10:00:00+00:00"
    },
    {
      "student_id": 4,
      "student_name": "Bob Student",
      "student_email": "bob@school.test",
      "cohort_id": 1,
      "cohort_name": "Spring 2026 Cohort A",
      "status": "enrolled",
      "enrolled_at": "2026-03-06T11:00:00+00:00"
    }
  ],
  "meta": {
    "current_page": 1,
    "last_page": 1,
    "per_page": 15,
    "total": 2
  }
}
```

Note: Data is fetched from the Enrolment Service's enrolments endpoint filtered by `experience_id`. If the Enrolment Service is unavailable, returns empty data (graceful degradation). The `meta.total` represents the total number of individual student records.

Experience Student Detail (Screen 302)

```
GET /api/school/experiences/{id}/students/{studentId}
```

Returns detail for a specific student within an experience context. The lookup queries the Enrolment Service using both `student_id` and `experience_id` filters to find the student's actual enrolment record.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "student_id": 3,
```

```
  "student_name": "Alice Student",
  "student_email": "alice@school.test",
  "experience_id": 1,
  "cohort_id": 1,
  "cohort_name": "Spring 2026 Cohort A",
  "status": "enrolled",
  "enrolled_at": "2026-03-05T10:00:00+00:00",
  "credits": {
    "earned": 0,
    "total": 0,
    "progress": 0.0
  }
}
```

Error (404):

```
{
  "error": true,
  "message": "Student not found in this experience",
  "code": "NOT_FOUND"
}
```

Integration dependency: Credit data (`credits.earned`, `credits.total`, `credits.progress`) is stubbed at zero. Real credit/progress data depends on Team Papa's Course Service. See the "Integration Requests" section below.

Export Experience Students (CSV)

```
GET /api/school/experiences/{id}/students/export
```

Downloads a CSV file of students enrolled in this experience, with one row per student.

Headers:

- `Authorization: Bearer {token}` (required)

Response: Streamed CSV download (`Content-Type: text/csv`, filename: `experience-students.csv`)

CSV Columns:

Column	Description
`student_name`	Student's full name
`student_email`	Student's email address
`cohort_name`	Name of the cohort the student is enrolled in
`status`	Enrolment status (`enrolled` or `removed`)
`enrolled_at`	ISO 8601 timestamp of enrolment

Note: Student data is fetched from the Enrolment Service filtered by `experience_id`. If the Enrolment Service is unavailable, the export will contain only the header row.

Experience Contents (Screen 302)

```
GET /api/school/experiences/{id}/contents
```

Returns the course contents and block structure for an experience.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "experience_id": 1,
  "courses": [
    {
      "id": 101,
      "name": "Business Planning",
      "sequence": 1,
      "blocks": [
        {
          "id": 1,
          "name": "Introduction to Business Plans",
          "type": "lesson"
        },
        {
          "id": 2,
          "name": "Market Research Exercise",
          "type": "activity"
        }
      ]
    }
  ]
}
```

Integration dependency: Course data (names and blocks) comes from `MockCourseDataProvider` (implementing `CourseDataProviderInterface`). The block structure is hardcoded sample data. Real course data will come from Team Papa's Course Service. See the "Integration Requests" section below.

Experience Statistics (Screen 302)

```
GET /api/school/experiences/{id}/statistics
```

Returns aggregated enrolment and completion statistics for an experience.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "experience_id": 1,
  "enrolment": {
    "total_students": 12,
    "active": 10,
    "removed": 2
  },
  "completion": {
    "completed": 0,
    "in_progress": 10,
    "not_started": 0,
    "completion_rate": 0.83
  },
  "credit_progress": {
    "average": 0.0,
    "students_with_credits": 0
  }
}
```

Notes:

- `enrolment.removed` is now computed from the `removed_count` field returned by the Enrolment Service's cohort data.

- `completion_rate` is computed as `active / total_students`.

Integration dependency: `completion.completed`, `completion.not_started`, `credit_progress.average`, and `credit_progress.students_with_credits` are always `0`. Real progress tracking depends on Team Papa's Course Service. See the "Integration Requests" section below.

Dashboard Service (port 8001)

Base URL: `http://localhost:8001/api/school`

The Dashboard Service owns no database tables. It aggregates data from the Experience Service and Enrolment Service via HTTP calls. If a downstream service is unavailable, the dashboard returns degraded data (zeros/empty arrays) rather than failing.

Health Check

```
GET /api/school/dashboard/health
```

No authentication required.

Response (200):

```
{
  "status": "ok",
  "service": "dashboard",
  "timestamp": "2026-03-12T14:30:00+00:00"
}
```

Dashboard Overview

```
GET /api/school/dashboard
```

Returns the full aggregated dashboard for the authenticated school admin.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "school": {
    "id": 1,
    "name": "Hatchloom Academy"
  },
  "summary": {
    "problems_tackled": 0,
    "active_ventures": 1,
    "students": 25,
    "experiences": 3,
    "credit_progress": 0.0,
    "timely_completion": 0.0
  },
  "cohorts": {
    "active": 2,
    "completed": 1,
    "upcoming": 1,
    "total": 4
  },
  "students": {
    "total_enrolled": 18,
    "active_in_cohorts": 15,
    "not_assigned": 10
  },
```

```
  "statistics": {
    "enrolment_rate": 0.72,
    "average_completion": 0.0,
    "average_credit_progress": 0.0
  },
  "warnings": [
    {
      "type": "unassigned_students",
      "message": "10 students are not assigned to any active cohort",
      "severity": "warning"
    }
  ]
}
```

Integration dependencies:

- `summary.problems_tackled`, `summary.credit_progress`, `summary.timely_completion` come from a mock progress provider (`MockStudentProgressProvider` implementing `StudentProgressProviderInterface`) and return `0` / `0.0`. Real data depends on Team Papa's Course Service.
- `statistics.average_completion` and `statistics.average_credit_progress` are always `0.0` until real progress data is available from Team Papa.
- If the Experience Service or Enrolment Service is down, their respective sections fall back to zero/empty values and a `service_degraded` warning is added.

Warning types:

- `service_degraded` (severity: `warning`) -- a downstream service was unreachable
- `unassigned_students` (severity: `warning`) -- forwarded from Enrolment Service statistics
- `capacity_warning` (severity: `info`) -- forwarded from Enrolment Service statistics

Student Drill-Down

```
GET /api/school/dashboard/students/{studentId}
```

Returns detailed data for a single student, combining enrolment and credential information.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "student": {
    "id": 3,
    "name": "Alice Student",
    "email": "alice@school.test"
  },
  "enrolments": [
    {
      "cohort_id": 1,
      "cohort_name": "Spring 2026 Cohort A",
      "experience_name": "Entrepreneurship Basics",
      "status": "enrolled",
      "enrolled_at": "2026-03-05T10:00:00+00:00"
    }
  ],
  "progress": {
    "courses_completed": 1,
    "courses_in_progress": 2,
```

```
    "overall_completion": 0.35
  },
  "credentials": [],
  "curriculum_mapping": []
}
```

Error (404):

```
{
  "error": true,
  "message": "Student not found",
  "code": "NOT_FOUND"
}
```

Notes:

- Enrolment data is fetched from the Enrolment Service's student detail endpoint (`/api/school/enrolments/students/{studentId}`), querying directly by student ID. If the Enrolment Service is unavailable, `enrolments` will be empty.

Integration dependencies:

- `progress` values are hardcoded (`courses_completed: 1`, `courses_in_progress: 2`, `overall_completion: 0.35`). Real progress data depends on Team Papa's Course Service.
- `credentials` and `curriculum_mapping` come from a mock provider (`MockCredentialDataProvider` implementing `CredentialDataProviderInterface`) and return empty arrays. Real data depends on Karl's credential engine.
- See the "Integration Requests" section below for details.

PoS Coverage (R3 Reporting)

```
GET /api/school/dashboard/reporting/pos-coverage
```

Returns Alberta Program of Studies curriculum coverage data per student.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "school_id": 1,
  "pos_areas": ["Business Studies", "CTF Design Studies", "CALM"],
  "student_coverage": [
    {
      "student_id": 3,
      "name": "Alice Student",
      "coverage": {
        "Business Studies": 0.0,
        "CTF Design Studies": 0.0,
        "CALM": 0.0
      }
    }
  ],
  "school_averages": {
    "Business Studies": 0.0,
    "CTF Design Studies": 0.0,
    "CALM": 0.0
  }
}
```

Integration dependency: All coverage values come from a mock progress provider (`MockStudentProgressProvider` implementing `StudentProgressProviderInterface`) and are `0.0`. Real data depends on Karl's credential engine (PoS curriculum mappings) and Team Papa's Course Service (completion data). See the "Integration Requests" section below.

Engagement Rates (R3 Reporting)

```
GET /api/school/dashboard/reporting/engagement
```

Returns student engagement metrics for the last 30 days.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "school_id": 1,
  "period": "last_30_days",
  "school_averages": {
    "login_frequency": 0.0,
    "content_interaction": 0.0,
    "assignment_completion": 0.0
  },
  "student_engagement": [
    {
      "student_id": 3,
      "name": "Alice Student",
      "login_frequency": 0.0,
      "content_interaction": 0.0,
      "assignment_completion": 0.0
    }
  ]
}
```

Integration dependency: All engagement values come from a mock progress provider (`MockStudentProgressProvider` implementing `StudentProgressProviderInterface`) and are `0.0`. Real engagement metrics (login frequency, content interaction, assignment completion) depend on Team Papa's Course Service. See the "Integration Requests" section below.

All Widgets

```
GET /api/school/dashboard/widgets
```

Returns all dashboard widgets in a single response, built using the Factory Method pattern.

Headers:

- `Authorization: Bearer {token}` (required)

Response (200):

```
{
  "widgets": [
    {
      "type": "cohort_summary",
      "data": { }
    },
    {
      "type": "student_table",
      "data": { }
    },
```

```
    {
      "type": "engagement_chart",
      "data": { }
    }
  ]
}
```

Note: The exact widget types and their `data` payloads depend on the registered widgets in `DashboardWidgetFactory`. Each widget computes its own data from the shared context (school info, experiences, downstream service responses).

Single Widget

```
GET /api/school/dashboard/widgets/{type}
```

Returns a single dashboard widget by type name.

Headers:

- `Authorization: Bearer {token}` (required)

Path Parameters:

Parameter	Type	Description
`type`	string	Widget type (e.g., `cohort_summary`, `student_table`, `engagement_chart`)

Response (200):

```
{
  "type": "cohort_summary",
  "data": { }
}
```

Error (422) -- Unknown widget type:

```
{
  "error": true,
  "message": "Unknown widget type: invalid_type",
  "code": "VALIDATION_ERROR"
}
```

Integration Requests -- What We Need From Other Teams

This section documents the external data that Team Delta currently mocks or stubs, organized by the team responsible for providing it. Each item specifies the mock class we would replace, the interface already defined in our codebase, the response shape we need, and which of our endpoints depend on it.

Team Papa (Course Service)

1. Course Catalogue API

We need a REST endpoint to fetch the full course catalogue (course IDs, names, and block/lesson structures).

What we currently mock: `MockCourseDataProvider` in `experience-service/app/DataProviders/MockCourseDataProvider.php`, which returns 5 hardcoded courses with static block structures.

Interface already defined: `CourseDataProviderInterface` in `experience-service/app/Contracts/CourseDataProviderInterface.php` with methods: `getAllCourses()`, `getCourse(int $id)`, `courseExists(int $id)`, `getCoursesByIds(array $ids)`.

What we would replace it with: An `HttpCourseDataProvider` that calls your API. We just need to know the URL and response shape.

Suggested endpoint: `GET /api/courses`

Ideal response shape:

```
{
  "data": [
    {
      "id": 101,
      "name": "Business Planning",
      "blocks": [
        {
          "id": 1,
          "name": "Introduction to Business Plans",
          "type": "lesson"
        },
        {
          "id": 2,
          "name": "Market Research Exercise",
          "type": "activity"
        }
      ]
    }
  ]
}
```

Our endpoints that depend on this:

- `POST /api/school/experiences` -- validates `course_ids` against the catalogue
- `PUT /api/school/experiences/{id}` -- validates updated `course_ids`
- `GET /api/school/experiences/{id}` -- resolves course names for display
- `GET /api/school/experiences/{id}/contents` -- displays course blocks and lesson structure

2. Student Course Progress Data

We need per-student progress information: courses completed, courses in progress, completion percentage, and blocks completed.

What we currently mock: `MockStudentProgressProvider` in `dashboard-service/app/DataProviders/MockStudentProgressProvider.php`, which returns hardcoded zeroes for all progress metrics.

Interface already defined: `StudentProgressProviderInterface` in `dashboard-service/app/Contracts/StudentProgressProviderInterface.php` with methods: `countProblemsTackled()`, `calculateCreditProgress()`, `calculateTimelyCompletion()`, `getPosCoverage()`, `getEngagementRates()`.

Suggested endpoint: `GET /api/students/{studentId}/progress`

Ideal response shape:

```
{
  "student_id": 3,
```

```
  "courses_completed": 2,
  "courses_in_progress": 1,
  "overall_completion": 0.65,
  "blocks_completed": 12,
  "blocks_total": 20,
  "last_activity_at": "2026-03-10T14:00:00+00:00"
}
```

Our endpoints that depend on this:

- `GET /api/school/dashboard` -- `summary.credit_progress`, `statistics.average_completion`, `statistics.average_credit_progress`
- `GET /api/school/dashboard/students/{studentId}` -- `progress.courses_completed`, `progress.courses_in_progress`, `progress.overall_completion`
- `GET /api/school/experiences/{id}/students/{studentId}` -- `credits.earned`, `credits.total`, `credits.progress`
- `GET /api/school/experiences/{id}/statistics` -- `completion.completed`, `completion.not_started`, `credit_progress`

3. Engagement/Activity Metrics

We need per-student engagement data: login frequency, activity counts, and time-on-platform.

What we currently mock: Part of `MockStudentProgressProvider` (same class as above). The `getEngagementRates()` method returns zeroes for all students.

Suggested endpoint: `GET /api/students/{studentId}/engagement` or a bulk endpoint `GET /api/schools/{schoolId}/engagement`

Ideal response shape (per-student):

```
{
  "student_id": 3,
  "login_frequency": 4.5,
  "content_interaction": 12.0,
  "assignment_completion": 0.85,
  "period": "last_30_days"
}
```

Our endpoints that depend on this:

- `GET /api/school/dashboard/reporting/engagement` -- per-student and school-average engagement metrics
- `GET /api/school/dashboard/widgets/engagement_chart` -- engagement widget data

Karl (Credential Engine)

1. Student Credential Data

We need per-student credential information: credentials earned, in-progress, and their details.

What we currently mock:

- `MockCredentialDataProvider` in `dashboard-service/app/DataProviders/MockCredentialDataProvider.php` -- returns sample mock data for `getStudentCredentials()` and `getStudentCurriculumMapping()`.

- `MockCredentialDataProvider` in `enrolment-service/app/DataProviders/MockCredentialDataProvider.php` -- returns `{"total_earned": 0, "in_progress": 0, "details": []}` for `getStudentCredentialSummary()`.

Interfaces already defined:

- `CredentialDataProviderInterface` in `dashboard-service/app/Contracts/CredentialDataProviderInterface.php` with methods: `getStudentCredentials(int $studentId)`, `getStudentCurriculumMapping(int $studentId)`.
- `CredentialDataProviderInterface` in `enrolment-service/app/Contracts/CredentialDataProviderInterface.php` with method: `getStudentCredentialSummary(int $studentId)`.

Suggested endpoint: `GET /api/students/{studentId}/credentials`

Ideal response shape:

```
{
  "student_id": 3,
  "total_earned": 2,
  "total_in_progress": 1,
  "credentials": [
    {
      "id": 1,
      "name": "Business Fundamentals Certificate",
      "type": "certificate",
      "status": "earned",
      "awarded_at": "2026-03-01T12:00:00+00:00"
    },
    {
      "id": 2,
      "name": "Digital Marketing Badge",
      "type": "badge",
      "status": "in_progress",
      "awarded_at": null
    }
  ]
}
```

Our endpoints that depend on this:

- `GET /api/school/dashboard/students/{studentId}` -- `credentials` array
- `GET /api/school/enrolments/students/{studentId}` -- `credentials.total_earned` and `credentials.details`

2. PoS (Program of Study) Curriculum Mappings

We need per-student Alberta Program of Studies coverage data: which curriculum areas have been covered and percentage coverage per area.

What we currently mock: Part of `MockStudentProgressProvider` and `MockCredentialDataProvider`. The `getPosCoverage()` method returns `0.0` for all PoS areas, and `getStudentCurriculumMapping()` returns an empty array.

Suggested endpoint: `GET /api/students/{studentId}/curriculum-mapping`

Ideal response shape:

```
{
```

```
  "student_id": 3,
  "coverage": {
    "Business Studies": 0.45,
    "CTF Design Studies": 0.30,
    "CALM": 0.15
  },
  "details": [
    {
      "area": "Business Studies",
      "outcomes_covered": 9,
      "outcomes_total": 20
    }
  ]
}
```

Our endpoints that depend on this:

- `GET /api/school/dashboard/reporting/pos-coverage` -- per-student and school-average PoS coverage
- `GET /api/school/dashboard/students/{studentId}` -- `curriculum_mapping` array

Team Quebec (Auth Service)

1. Real JWT/Token-Based Authentication

We need to replace our mock authentication with real token verification.

What we currently mock: `MockAuthMiddleware` in each service (e.g., `dashboard-service/app/Http/Middleware/MockAuthMiddleware.php`). This middleware maps hardcoded bearer tokens to mock user records with `id`, `name`, `email`, `role`, and `school_id`.

What we need from you:

- The JWT/token format and signing mechanism so we can verify tokens.
- An endpoint or mechanism to validate a token and extract the user's identity.
- The token payload should include (or allow us to derive): `user_id`, `role` (one of `school_admin`, `school_teacher`, `student`), and `school_id`.

Suggested token payload:

```
{
  "sub": 1,
  "role": "school_admin",
  "school_id": 1,
  "exp": 1711900800
}
```

What we would replace: The `MockAuthMiddleware` in all three services (`dashboard-service`, `experience-service`, `enrolment-service`) would be replaced with a `JwtAuthMiddleware` that validates the token against your signing key or verification endpoint.

Our endpoints that depend on this: Every authenticated endpoint across all three services requires a valid bearer token. The `role` is used for authorization checks (only `school_admin` and `school_teacher` are allowed), and `school_id` is used to scope all data queries.

Error Format

All error responses follow a consistent structure:

```
{
```

```
  "error": true,
  "message": "Human-readable error message",
  "code": "ERROR_CODE"
}
```

Error codes used across all services:

Code	HTTP Status	Description
`UNAUTHENTICATED`	401	Missing or invalid bearer token
`FORBIDDEN`	403	Valid token but insufficient role
`NOT_FOUND`	404	Resource does not exist or is outside the caller's school
`INVALID_STATE_TRANSITION`	409	Cohort state change violates the lifecycle (`not_started -> active -> completed`)
`VALIDATION_ERROR`	422	Input validation failure or business rule violation
`DUPLICATE_ENROLMENT`	422	Student is already enrolled in the specified cohort

Note on Laravel validation errors: For request validation failures (missing/invalid fields), Laravel returns its own format:

```
{
  "message": "The name field is required.",
  "errors": {
    "name": ["The name field is required."]
  }
}
```

This differs from the custom error format above. Custom error responses are used for business logic errors.

Docker Network Access

When calling Team Delta's services from another Docker container on the same Docker network, use the **service names** as hostnames instead of `localhost`:

Service	Internal URL	External URL (host machine)
Dashboard Service	`http://dashboard-service:8001`	`http://localhost:8001`
Experience Service	`http://experience-service:8002`	`http://localhost:8002`
Enrolment Service	`http://enrolment-service:8003`	`http://localhost:8003`

All endpoints are prefixed with `/api/school/`. For example, to call the enrolment statistics from another container:

```
GET http://enrolment-service:8003/api/school/enrolments/statistics
Authorization: Bearer test-admin-token
```

Important: The internal hostnames (`dashboard-service`, `experience-service`, `enrolment-service`) are only resolvable within the Docker network. From the host machine or CI runners, use `localhost` with the appropriate port.

School Scoping

All data returned by Team Delta's services is automatically filtered by the authenticated user's `school_id`. This is enforced at the model level via a Laravel Global Scope (`SchoolScope`) on all models that have a `school_id` column.

What this means for consumers:

- You will only ever see data belonging to your school.
- You do not need to pass a `school_id` parameter -- it is derived from the bearer token.
- There is no way to query data from another school, even by guessing IDs. A resource ID that belongs to a different school will return `404 Not Found`.

Note: Since both test tokens map to `school_id = 1`, all test data belongs to school 1. Cross-school isolation can be verified by seeding a second school with its own token in tests.